Appello 09 luglio 2015



Politecnico di Milano

Anno accademico 2014-2015

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:
Esercizio 2:
Esercizio 3:
Esercizio 4:

Esercizio 1

Si consideri un'astrazione sui dati che definisce un conto corrente bancario (classe `CCB`), che fornisce le operazioni seguenti:

- creazione di un `CCB`, che crea un conto corrente con saldo 0;
- deposito nel `CCB`, che ha per parametro un valore `float` rappresentante il valore depositato;
- prelievo dal `CCB`, che ha per parametro un valore `float` che rappresenta il valore da prelevare. L'operazione può avvenire soltanto se il conto corrente non va in rosso;
- calcolo del saldo del `CCB`;
- calcolo dei movimenti effettuati: viene prodotto l'elenco di tutte le operazioni fatte sul `CCB` dalla sua creazione in ordine cronologico. Ogni movimento è un valore positivo se si è trattato di un deposito, negativo se di un prelievo.

Si consideri il seguente scheletro di specifica della classe `CCB`:

```
public class CCB {
    public CCB(){...}
    public List<Float> movimenti(){...}
    public void deposito(float x){...}
    public void prelievo (float x){...}
    public float saldo(){...}
}
```

Domanda a

Precisare a fianco di ciascun metodo della classe se esso è puro. Si fornisca la specifica JML in termini di pre- e post-condizioni delle operazioni `CCB`, `prelievo` e `saldo`, cercando se possibile di definire operazioni **totali**.

SOLUZIONE: Sono `/*@ pure @*/` i metodi `saldo(), movimenti()`. Non è invece necessario (pur se non errato) specificare come puro il costruttore CCB (in quanto non ha parametri e non serve richiamarlo nella specifica di altri metodi).

Per quanto non richiesto dal testo, è utile a fini didattici mostrare i contratti anche dei metodi `movimenti()` e `deposito`. Il metodo `movimenti()` è l'unico metodo puro "primitivo", ossia che deve essere specificato con un commento. Ri fatto restituisce tutte le informazioni necessarie per specificare gli altri metodi.

```
//@ensures \result \neq null && (*\result e' una lista di
//@ tutte le operazioni di deposito o prelievo su this in ordine cronologico*);
public /*@pure@*/ List<Float> movimenti()

//@ensures movimenti().size()==0;
public CCB()

//@ensures \result ==
//@ (\sum int i; 0 <=i && i < movimenti().size(); movimenti().get(i));
public /*@pure@*/ float saldo()
```

NB: L'operatore `\sum` restituisce 0 quando il suo range è vuoto. Quindi quando `movimenti().size()==0` necessariamente è `saldo()==0`. Non è comunque errato (ma soltanto inutile) aggiungere alla postcondizione di `CCB` la condizione `saldo()==0`.

```
//@ensures x>0 && \old(saldo()>=x) &&
//@  movimenti().size() == 1+\old(movimenti().size()) &&
```

```
//@  movimenti().get(movimenti().size()-1) == -x &&
//@  movimenti().sublist(0,movimenti().size()-2).equals(\old(movimenti());
//@signals (NegativeException e) (x<0 || \old(saldo()<x)) &&
//@  movimenti().equals(\old(movimenti()));
public void prelievo (float x) throws NegativeException;

//@ensures x>0 &&
//@  movimenti().size() == 1+\old(movimenti().size()) &&
//@  movimenti().get(movimenti().size()-1) == x &&
//@  movimenti().sublist(0,movimenti().size()-2).equals(\old(movimenti());
//@signals (NegativeException e) x<0 && movimenti().equals(\old(movimenti()));
public void deposito (float x) throws NegativeException;
```

Domanda b

Specificare l'invariante astratto che il saldo è sempre non negativo. Mostrare che questa proprietà è effettivamente garantita dalla specifica fornita delle operazioni.

SOLUZIONE:

`public invariant saldo()>=0;`

Il costruttore verifica l'invariante in quanto definisce un CCB con `saldo=0`. Mostriamo che se l'invariante vale al momento della chiamata dei metodi non puri di CCB allora vale anche all'uscita. Ci sono solo due metodi non puri: deposito: può solo aumentare il saldo (a patto che sia stato specificato correttamente il lancio di un'eccezione se l'argomento è negativo); prelievo: la sua post-condizione prevede `\old(saldo()>=x)`, pertanto `\old(saldo()) + movimenti().get(size()-1) >=0`. Il nuovo saldo() al termine sarà uguale alla somma di tutti i nuovi movimenti, ossia la somma di quelli vecchi (uguale a `\old(saldo))` più il nuovo movimento uguale a -x, ossia proprio:

`\old(saldo())+ movimenti().get(size()-1) == saldo()` con `x>=0`.

Domanda c

Si consideri una rep costituita da

1. un `ArrayList<Float> lista` in cui sono memorizzate, in ordine cronologico, tutte le operazioni di prelievo o deposito.
2. un `ArrayList<Long> tempo`, nella cui posizione i-esima è memorizzato (come un numero `Long` positivo) l'istante di tempo in cui è stata effettuata l'operazione i-esima dalla creazione.

- Si scriva l'invariante di rappresentazione della classe e la funzione di astrazione (scegliendo liberamente per quest'ultima la notazione da usare).

  SOLUZIONE:

  RI:

```
private invariant lista!= null && tempo!=null &&
  !lista.contains(null) && !tempo.contains(null) &&
  lista.size()== tempo.size() &&
  (\forall int i; 0 <=i && i<tempo.size()-1;
           0<tempo(i) && tempo.get(i) < tempo. get(i+1));
```

  AF:

```
private invariant movimenti().equals(lista);
```

  Una soluzione equivalente consiste nel reiplementare il metodo `toString()`

- Si mostri un'implementazione scorretta del metodo `movimenti` che esponga la *rep* e si spieghi perché ciò sia assolutamente da evitare.

SOLUZIONE:

```
return lista;
```

Il metodo espone la parte mutabile della rep, consentendo al codice cliente di modificare direttamente lo stato concreto di un oggetto, aggirando information hiding, e causando potenziali violazioni del RI tramite effetti collaterali.

Esercizio 2

Si consideri questo frammento di codice Java:

```
class Cane{
   public void abbaia(){
      System.out.println("bau");
   }

   public void abbaia(int num){
      for(int i=0; i<num; i++)
         System.out.println("bau");
   }
}

class CaneDaCaccia extends Cane{
   public void fiuta(){
      System.out.println("ffff");
   }

   public void abbaia(){
      System.out.println("woo");
   }
}

-------

Cane fido = new Cane();
CaneDaCaccia pluto = new CaneDaCaccia();
Cane febo = new CaneDaCaccia();
CaneDaCaccia pippo = new Cane();

fido.abbaia();
pluto.abbaia();
febo.abbaia();
pippo.abbaia();
fido.abbaia(3);
pluto.abbaia(3);
febo.abbaia(3);
pippo.abbaia(3);
```

- Marcare con `E` le istruzioni dopo `----` che generano un errore in compilazione.
- A fianco di ciascuna chiamata di metodo che non generi errore, si scriva quale sarebbe il valore stampato in uscita.

SOLUZIONE:

```
CaneDaCaccia pippo = new Cane(); //ERRATA

fido.abbaia(); //bau
pluto.abbaia(); //woo
febo.abbaia(); //woo
pippo.abbaia(); //pippo non può essere inizializzato
fido.abbaia(3); //bau bau bau
pluto.abbaia(3); //bau bau bau
febo.abbaia(3); //bau bau bau
pippo.abbaia(3); //pippo non può essere inizializzato
```

Esercizio 3

Si consideri la seguente specifica di un metodo statico:

```
//@ requires true;
//@ ensures \result <==> (\exists int i; 1<=i && i <elements.size();
//@    elements.get(i).equals(elements.get(i-1)) ||
//@  (elements.get(i).equals(elements.get(i-2)) &&  i>1))
//@ signals (BadArgumentException e) elements==null ||
//@    (\exists int i; 0<=i && i < elements.size(); elements.get(i)== 0)
public static /*@pure @*/ boolean twoNext(ArrayList<Integer> elements)
throws BadArgumentException
```

Definire un insieme di casi di test secondo una strategia *black-box* e fornire un sintetico commento che giustifichi perché si ritiene significativo questo insieme.

Nel testing black box, la specifica fornisce uno strumento che consente di partizionare il dominio di input. Una volta partizionato il dominio è necessario:

- testare ogni categoria (Metodo delle combinazioni);
- testare i valori ai confini delle categorie (Metodo dei casi limite).

È necessario come prima cosa andare ad identificare le categorie di interesse utilizzando la specifica.
Iniziamo con l'analisi della postcondizione normale:

```
//@ (\exists int i; 1<=i && i <elements.size();
//@    elements.get(i).equals(elements.get(i-1)) ||
//@  (elements.get(i).equals(elements.get(i-2)) &&  i>1))
```

è vera in due condizioni:

- `\exists int i; 1<=i && i <elements.size();`
  `elements.get(i).equals(elements.get(i-1))`

  esiste un elemento uguale al suo predecessore (condizione A).
- `\exists int i; 1<=i && i <elements.size();`
  `(elements.get(i).equals(elements.get(i-2)) &&  i>1))`

  esiste un elemento uguale al predecessore del suo predecessore (condizione B).

Possiamo a questo punto analizzare la post-condizione eccezionale anch'essa fatta da due clausole:

- `elements==null`

  elements è nullo. (condizione D)
- `(\exists int i; 0<=i && i < elements.size(); elements.get(i)== 0)`

  elements contiene un valore zero (condizione C)

Partiamo con il *test di categoria*. Il primo test associa a elements il valore `null` rende soddisfatta la condizione D.

Si noti che la Condizione D e' mutuamente esclusiva con A, B e C. C invece non è mutuamente esclusiva con A e B e pertanto occorre considerare tutte le possibilità.

A questo punto, date le condizioni A, B, e C è possibile identificare otto sottocategorie a seconda che A, B e C siano soddisfatte. Partendo da queste sottocategorie il seguente *test di categoria* può essere delineato:

A	B	C	Test
Vera	Vera	Vera	[1, 2, 3, 4, 1, 1, 1, 0, 4]
Vera	Vera	Falsa	[1, 2, 3, 4, 1, 1, 1, 6, 4]
Vera	Falsa	Vera	[1, 2, 3, 4, 1, 1, 9, 0, 4]
Vera	Falsa	Falsa	[1, 2, 3, 4, 1, 1, 9, 6, 4]
Falsa	Vera	Vera	[1, 2, 3, 4, 1, 0, 1, 6, 4]
Falsa	Vera	Falsa	[1, 2, 3, 4, 1, 10, 1, 6, 4]
Falsa	Falsa	Vera	[1, 2, 3, 4, 1, 10, 14, 6, 4]
Falsa	Falsa	Falsa	[1, 2, 3, 4, 1, 10, 16, 6, 4]

Focalizziamoci sui *test al confine*. Occorre testare il metodo almeno per i seguenti casi:

- array vuoto [];
- array con un elemento [1];

Inoltre sarebbe utile testare casi in cui le condizioni che rendono vera/falsa la specifica sono posizionate "agli estremi dell'`ArrayList`, per esempio [1, 1, 10,20, 8, 9] e [6, 7, 8, 9, 8, 1 1] sono test di confine per la condizione A, [1, 6, 1,20, 8, 9] e [2, 6, 4,1, 8, 1] sono test di confine per la condizione B e [0], [0,2], [1,2,0] sono test di confine per la condizione C.

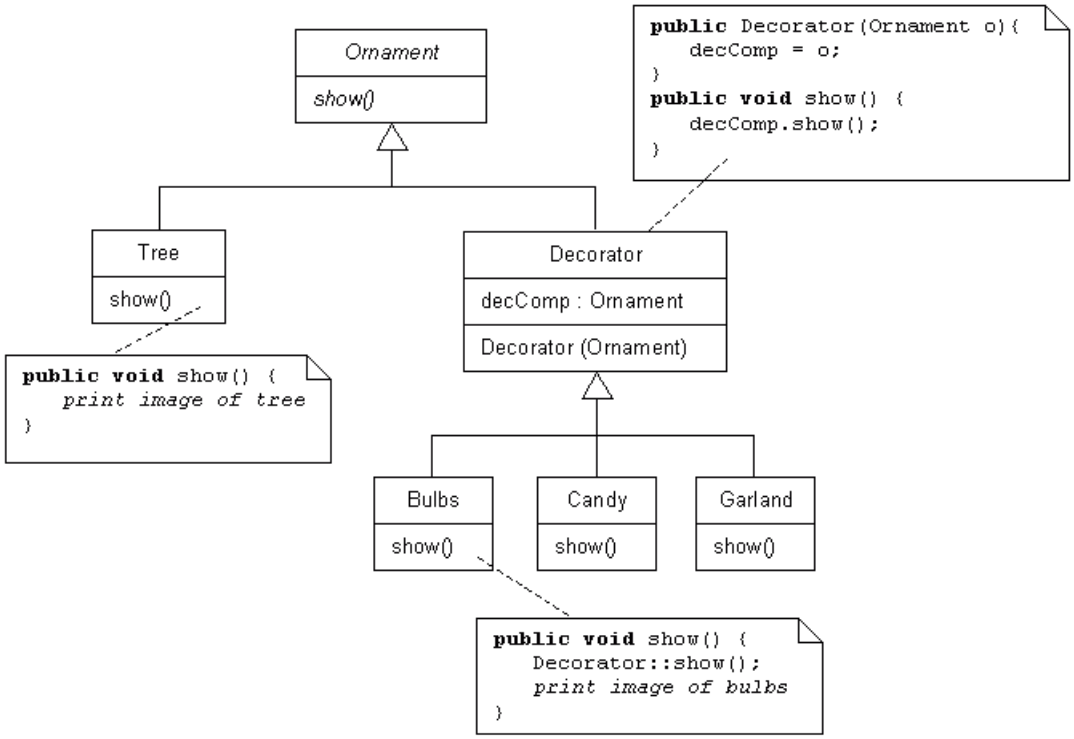
Esercizio 4

Si usi il design pattern *Decorator* per modellare un `AlberoDiNatale`, come componente di base, e le sue solite decorazioni (`Decorazione`) —ad esempio, `Luci`, `Ghirlanda` e `Dolcetto`. Si assuma che ogni tipo di decorazione è aggiunto una sola volta. `AlberoDiNatale` ha un metodo `show()` che disegna l'albero sullo schermo. Ogni decorazione ha pure un metodo `show()` che disegna la decorazione stessa. Attraverso il pattern *Decorator*, l'albero completo viene disegnato attraverso l'invocazione del metodo `show()` della classe `AlberoDiNatale` opportunamente gestito attraverso la gerarchia di classi definita.

Si realizzi il diagramma UML delle classi per la soluzione proposta e lo si esemplifichi con un paio di configurazioni significative. Si mostri in particolare che cosa accade invocando il metodo `show()` su un oggetto di tipo `AlberoDiNatale` che è stato decorato, nell'ordine, con i decoratori `Luci`, `Ghirlanda` e `Dolcetto`; si ipotizzi per semplicità che ciascun metodo `show()` stampi semplicemente il nome della classe in cui è definito.

SOLUZIONE

Il pattern decorator permette di incapsulare un oggetto all'interno di un altro oggetto che ne "decora" il comportamento. Il decorator ha la stessa interfaccia del componente che decora per cui la sua presenza è trasparente a un client. Il decorator chiama i metodi dell'oggetto che incapsula, ma li può appunto decorare eseguendo operazioni addizionali. Il fatto che il decoratore sia trasparente permette di includere il decoratore in un altro decoratore ricorsivamente (una specie di matrioska di decoratori) in cui ogni decoratore decora le funzionalità di quello che contiene Nel caso in questione, il componente base è di tipo `AlberoDiNatale`,



sottoclasse della classe astratta `Ornamento`. Questa classe, possiede un metodo `show()`, che stampa le caratteristiche dell'`Ornamento`. È possibile decorare il metodo `show` dell'albero mediante i vari ornamenti che possono essere aggiunti via via all'albero. Nel caso specifico le decorazioni, sottoclassi della classe astratta `Decorazione` (a sua volta sottoclasse di ornamento), includono `Luci`, `Ghirlanda` e `Dolcetto`. Ogni decorazione ha un reference all'`Ornamento` decorato. Inoltre, le decorazioni possono contenere attributi addizionali per esempio le `Luci` potrebbero avere un colore, i dolcetti potrebbero avere un gusto etc. Quando viene invocato il metodo `show` su una decorazione, il metodo `show` del componente contenuto viene chiamato, e il risultato viene decorato con il nuovo componente. Per esempio, nel caso delle `Luci` viene

effettuata una stampa che specifica che l'albero è decorato per mezzo di luci di un determinato colore, mentre nel caso dei `Dolcetti` la decorazione e' composta da Dolcetti di un dato sapore.

Due oggetti `o1 e o2` di tipo `Ornamento`, di cui il secondo specificato come richiesto (un `AlberoDiNatale` decorato con `Luci`, `Ghirlanda` e `Dolcetto`), possono essere definiti come segue:

```
Ornamento o1=new (Ghirlanda(new Luci(Color.RED, new Dolcetto("Menta",
    AlberoDiNatale())));
Ornamento o2=new Dolcetto("Limone", new Ghirlanda(new Luci(Color.BLUE, new
    AlberoDiNatale())));
o2.show();
```

Quando viene chiamato il metodo `show` dell'ornamento ogni componente richiama il metodo show dell'elemento contenuto e lo decora mediante il suo risultato. Nel caso in questione un possibile output dell'invocazione del metodo `show` sull'oggetto `o2` è

```
AlberoDiNatale\\
Luci Blu\\
Ghirlanda\\
Dolcetto al Limone\\
```